

DE 424 Mini Project - Find the Wumpus

A Probabilistic Graphical Model for Wumpus Tracking

Kehan Taljaard 26867206

14 May 2026

Declaration

I, Kehan Taljaard, declare that this report and the work presented in it are my own work, except where explicitly acknowledged below. I have not received or accepted information from any source not properly acknowledged in this declaration or elsewhere in the report.

Use of AI assistance. An AI coding assistant (Claude) was used in a limited, supporting capacity for this project. Its use was restricted to two areas. First, transcribing and typesetting: the model derivations, the design reasoning and the analysis in this report were worked out by hand on paper, and Claude was used to help convert that handwritten work into LaTeX and to tidy the document formatting. Second, plotting: Claude helped write parts of the Python scripts used to create the figures from the result data. It was not used to derive the model, implement the core `emdw` inference or EM code, or interpret the results. Claude was also given the full `emdw` library codebase and wrote a reference manual that I used often while writing the implementation, as it contained good `emdw` and C++ practices and explanations. All derivations, modelling and design decisions, code implementation, and the analysis of all results are my own work. I also used Claude to setup building so that I could run my code outside of `/devel/emdw` and have it work and build in a separate folder.

Other sources. The DE 424 lecture notes (Weeks 1–10), the *emdw* user guide and the example code provided with the project, and the textbook by Barber [1] (in particular Chapter 23 on dynamic models) were consulted for theoretical background and library API usage.

ECSA Graduate Attributes

GA 1 - Problem solving. I formulated the wumpus tracking problem as a hidden Markov model, using a uniform random-walk transition between cells and a per-cell Bernoulli detection model (Section 2). This structure was chosen because the wumpus moves according to Markov dynamics, and because the per-cell detections are conditionally independent given the wumpus position. I derived the model by hand, with guidance from Barber’s work on hidden Markov models [1, Ch. 23]. The main simplification, in which the $(T+1)G$ binary detection variables are collapsed into $T+1$ unary evidence factors on the wumpus positions, is derived in Section 2.5. The main design trade-offs, specifically learning p_w and p_c as point estimates by EM rather than by full Bayesian inference, and restricting the work to datasets 1–3, are discussed in Sections 2.8 and 4.

GA 2 - Application of scientific and engineering knowledge. The solution applies probability theory (joint factorisation, marginalisation and conditional independence), graphical-model theory (Bayes networks, factor graphs and cluster graphs) and the relevant inference algorithms (sum-product belief propagation and the Expectation–Maximisation algorithm) to the engineering problem of tracking a target from noisy measurements. The C++ implementation, built on the

emdwl library, realises this design (Section 3). The convergence behaviour of the EM parameter learning is reported in Section 4.

1 Problem description

A wumpus performs a random walk on a $W \times H$ grid over $T+1$ discrete time steps. Its position is unobserved. At each time step, every cell of the grid emits a binary detection: the wumpus’s cell emits a 1 with probability p_w and a 0 otherwise, while every other cell emits a 1 with probability p_c and a 0 otherwise. The entire detection grid is observed at every time step. The task is to infer the wumpus’s trajectory from the sequence of detection grids.

Five datasets are provided. This work addresses datasets 1, 2 and 3. The scoping decision is motivated in Section 2.8.

2 Solution design

2.1 Notation

Symbol	Meaning
W, H	grid width and height (columns and rows)
$G = W \cdot H$	total number of cells
T	final time step index ($T+1$ steps in total)
$\mathcal{X}$	state space, $\{0, 1, \dots, G-1\}$
$X_t \in \mathcal{X}$	unobserved wumpus cell index at time t
$D_t^{(i)} \in \{0, 1\}$	detection RV at cell i at time t
$d_t^{(i)} \in \{0, 1\}$	observed value of $D_t^{(i)}$
$\mathbf{d}_t$	full detection grid at time t
p_w, p_c	detection probabilities (wumpus cell / other cells)
$\gamma_t(x)$	smoothed posterior $P(X_t = x \mid \mathbf{d}_{0:T})$

2.2 Random variables and Bayes network structure

The unobserved variables are the wumpus positions $X_0, \dots, X_T$, each a discrete RV over the G cells under a position encoding $i = \text{row} \cdot W + \text{col}$. The observed variables are the detections $D_t^{(i)}$ for each cell i and time t . The conditional independence assumptions encoded by the model are (from [[1], Ch. 23]) the following:

- **Markov dynamics:** $X_{t+1} \perp\!\!\!\perp X_{0:t-1} \mid X_t$.
- **Conditionally independent detections:** given X_t , the per-cell detections at time t are mutually independent of each other and of all detections at other times.

The resulting Bayes net is shown in Figure 1. The joint distribution factorises as

$$p(X_{0:T}, \{D_t^{(i)}\}) = p(X_0) \prod_{t=0}^{T-1} p(X_{t+1} \mid X_t) \prod_{t=0}^T \prod_{i=0}^{G-1} p(D_t^{(i)} \mid X_t). \quad (1)$$

2.3 Initial-state prior

Due to a lack of information about the wumpus’s starting cell, a uniform prior is used:

$$p(X_0 = x) = \frac{1}{G}, \quad \forall x \in \mathcal{X}. \quad (2)$$

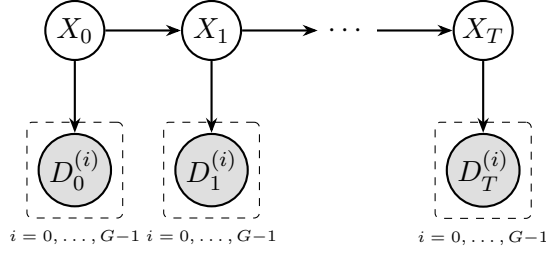


Figure 1: Bayes network for the wumpus tracking model. Each unobserved X_t has G binary detection children.

2.4 Transition factor

At each step the wumpus selects a direction $d \in \{U, D, L, R\}$ uniformly and attempts a move. If the target cell is out of grid bounds the move fails and the wumpus remains in place. Marginalising out the unobserved direction, the transition factor is given by:

$$p(X_{t+1} = j | X_t = i) = \frac{1}{4} \sum_{d \in \{U, D, L, R\}} \mathbb{I}[\text{move}(i, d) = j]$$

where $\text{move}(i, d)$ returns the target cell of move d from i , clipped back to i if the target is out of bounds.

The factor is sparse: each row has at most five non-zero entries (at most four in-bounds neighbours plus the cell's own mass carried back from clipped moves). The factor is time-homogeneous, so a single sparse table is built at the beginning and used for every transition.

2.5 Observation factor collapse

Because every detection $D_t^{(i)}$ is observed, the $(T+1)G$ emission factors can be folded into one unary factor per time step on X_t , removing the detection RVs from the graph entirely.

By the conditional independence of detections given the wumpus position (Section 2.2), the joint emission likelihood at time t factorises as:

$$p(\mathbf{d}_t | X_t = x^*) = \prod_{i=0}^{G-1} p(D_t^{(i)} = d_t^{(i)} | X_t = x^*),$$

and each per-cell likelihood depends only on whether cell i contains the wumpus:

$$p(D_t^{(i)} = d_t^{(i)} | X_t = x^*) = \begin{cases} p_w^{d_t^{(i)}} (1-p_w)^{1-d_t^{(i)}} & i = x^*, \\ p_c^{d_t^{(i)}} (1-p_c)^{1-d_t^{(i)}} & i \neq x^*. \end{cases}$$

Next, we can factor out the $i \neq x^*$ terms, which do not depend on x^* and so:

$$p(\mathbf{d}_t | X_t = x^*) = p_w^{d_t^{(x^*)}} (1-p_w)^{1-d_t^{(x^*)}} \cdot \overbrace{\prod_{i \neq x^*} p_c^{d_t^{(i)}} (1-p_c)^{1-d_t^{(i)}}}^{(1)}$$

Where (1) is expanded to the following to remove terms independent of x^* :

$$\frac{\prod_{i=0}^{G-1} p_c^{d_t^{(i)}} (1-p_c)^{1-d_t^{(i)}}}{p_c^{d_t^{(x^*)}} (1-p_c)^{1-d_t^{(x^*)}}}$$

Now:

$$p(\mathbf{d}_t | X_t = x^*) = \underbrace{\left[\prod_{i=0}^{G-1} p_c^{d_t^{(i)}} (1-p_c)^{1-d_t^{(i)}} \right]}_{C_t, \text{ independent of } x^*} \cdot \frac{p_w^{d_t^{(x^*)}} (1-p_w)^{1-d_t^{(x^*)}}}{p_c^{d_t^{(x^*)}} (1-p_c)^{1-d_t^{(x^*)}}}.$$

C_t does not depend on x^* and cancels in any posterior over X_t . We define the unary observation factor:

$$\phi_t(X_t = x^*) \propto \begin{cases} p_w/p_c & d_t^{(x^*)} = 1, \\ (1-p_w)/(1-p_c) & d_t^{(x^*)} = 0. \end{cases}$$

This is a length- G table per time step. The $G(T+1)$ binary detection RVs are absent from the graph used for inference, which becomes $T+1$ unobserved variables with one unary evidence factor per step, a factor of G times smaller than the naive representation.

2.6 Cluster graph and inference algorithm

The factor list is passed to `emdw`'s `ClusterGraph` constructor with either the `LTRIP`, `BETHE` or `JTREE` construction tag. For the chain-structured factor graph in this model, a non-looping cluster graph is constructed, so sum-product belief propagation is exact and converges after one forward and one backward sweep. `loopyBP_CG` on this tree-structured CG behaves as standard forward-backward.

Sum-product marginal inference was chosen over MAP for two reasons:

- The per-time-step marginal $\gamma_t(x) = p(X_t | \mathbf{d}_{0:T})$ (i.e. the posterior given all the time steps, not just the preceding ones) is the obvious output for visualisation and for any further purpose of this algorithm. For example, any function that would need to know where the wumpus is would also probably need to know how likely the wumpus is to be there. Saying that the wumpus is in square 1 if it's only slightly less likely to be in square 2 would be misrepresenting vital information for any real world application of hunting wumpi (or wumpuses)
- the marginals $\gamma_t(x)$ are exactly the quantity required by the E-step of the EM algorithm used for dataset 3 (Section 2.7).

2.7 Parameter learning for unknown p_w, p_c (dataset 3)

For dataset 3, the detection parameters are unknown. We treat them as point estimates to be learned by the EM algorithm. This choice keeps the graph chain-structured (so inference remains exact in each E-step). The alternative of placing Beta priors on p_w, p_c and performing full Bayesian inference would have been possible but was not implemented because it would have caused loopy (and thus non-exact) inference due to the parameters being shared parents of all the observation factors.

The M-step is a Bernoulli MLE similar in structure to the Gaussian-mixture derivation in Week 10 of the class notes [2, Week 10]. With $A_t = \sum_x \gamma_t^{(k)}(x) d_t^{(x)}$ (the expected detection at the wumpus cell at time t) and $D_t = \sum_i d_t^{(i)}$ (the total detection count at time t), the closed-form updates are

$$p_w^{(k+1)} = \frac{\sum_t A_t}{T+1}, \quad p_c^{(k+1)} = \frac{\sum_t (D_t - A_t)}{(T+1)(G-1)}.$$

The algorithm alternates between E and M until both of the parameters change by less than 10^{-6} per iteration or 50 iterations are reached. Parameter estimates are clamped to $[10^{-6}, 1-10^{-6}]$ to avoid singular observation factors at boundary values. EM is initialised at $(p_w^{(0)}, p_c^{(0)}) = (0.9, 0.1)$.

2.8 Scope of solution

This work addresses datasets 1–3 in depth and does not extend to datasets 4–5. Choosing to focus on these datasets allowed for thorough analysis of datasets 1–3.

3 Implementation

The implementation consists of two main programs and a shared helper library:

- `miniproject.cc`: datasets 1–2 (known parameters)
- `miniproject_q3.cc`: dataset 3 (EM-based parameter learning)
- `miniproject_helpers.cc`: helper files for detection grids, ground truth, marginals, and trajectories

Both main programs share the same skeleton: load detections, build factors, construct cluster graph, run BP, extract per-time-step marginals, output results, but with the EM variant wrapping factor build, CG construction, BP, and marginal extraction in an outer loop implementing the M-step of Section 2.7. Each program takes the dataset directory, output directory, initial (p_w, p_c) , and an optional cluster-graph type (LTRIP, JTREE or BETHE) on the command line. The code was written with close reference to [3], and with reference to an LLM generated user guide that had full context from the emdw library files and guided me on proper C++ syntax and usage for emdw-specific cases.

3.1 Variable identifiers and shared cell domain

Each X_t is assigned a sequential integer identifier, and all X_t share a single length- G cell domain created once:

```
vector<RVIdType> X(lastT + 1);
for (int t = 0; t ≤ lastT; ++t) X[t] = static_cast<RVIdType>(t);

rcptr<vector<T>> cellDom(new vector<T>(G));
for (int i = 0; i < G; ++i) (*cellDom)[i] = i;
```

3.2 Building the factors

Prior. The uniform prior on X_0 is encoded by giving an empty map with a default probability of $1/G$, so every entry of the length- G table is automatically $1/G$:

```
map<vector<T>, FProb> emptyProbs;
rcptr<Factor> prior = uniqptr<DT>(new DT(
    {X[0]}, {cellDom}, 1.0 / static_cast<double>(G), emptyProbs));
```

Transition factor. The transition table is built by enumerating each source cell and moving $\frac{1}{4}$ of mass per direction into the right cell. Illegal moves are mapped back to the source cell by `clipMove`, which automatically generates the corner/edge self-loop mass:

```
auto clipMove =
[W, H, &cellOf](int row, int col, int drow, int dcol) {
    int nr = row + drow;
    int nc = col + dcol;
    if (nr < 0 || nr ≥ H) nr = row;
    if (nc < 0 || nc ≥ W) nc = col;
    return cellOf(nr, nc);
};
```

```

map<vector<T>, FProb> transProbs;
for (int i = 0; i < G; ++i) {
    const int r = rowOf(i);
    const int c = colOf(i);
    for (int d = 0; d < 4; ++d) {
        const int j = clipMove(r, c, directions[d][0], directions[d][1]);
        transProbs[{T(i), T(j)}].prob += 0.25;
    }
}

```

The sparse map is built once and reused across all T transition factors, using the fact that all the transitions use the same underlying structure.

Observation factor. Each ϕ_t takes one of the two values $\phi_{\text{on}} = p_w/p_c$ or $\phi_{\text{off}} = (1-p_w)/(1-p_c)$ per cell (see section 2.5), determined by the detection grid at time t :

```

const double phiOn = cfg.pw / cfg.pc;
const double phiOff = (1.0 - cfg.pw) / (1.0 - cfg.pc);

for (int t = 0; t ≤ lastT; ++t) {
    const Grid& det = detections[t];
    map<vector<T>, FProb> phiProbs;
    for (int row = 0; row < H; ++row) {
        for (int col = 0; col < W; ++col) {
            const int j = cellOf(row, col);
            const int d = det[row][col];
            phiProbs[{T(j)}] = (d == 1) ? phiOn : phiOff;
        }
    }
    rcptr<Factor> phi = uniqptr<DT>(new DT(
        {X[t]}, {cellDom}, defProb, phiProbs));
    factors.push_back(phi);
}

```

3.3 Cluster graph and BP

The cluster-graph construction algorithm is selectable at runtime via the command line, enabling the LTRIP/JTREE comparison (see Section 4):

```

map<RVIdType, AnyType> obsv;
if (cfg.cgType == "JTREE")
    cgPtr = new ClusterGraph(ClusterGraph::JTREE, factors, obsv);
else if (cfg.cgType == "BETHE")
    cgPtr = new ClusterGraph(ClusterGraph::BETHE, factors, obsv);
else
    cgPtr = new ClusterGraph(ClusterGraph::LTRIP, factors, obsv);

map<Idx2, rcptr<Factor>> msgs; MessageQueue msgQ;
unsigned nMsgs = loopyBP_CG(*cgPtr, msgs, msgQ);

```

3.4 Extracting marginals and the EM loop

For each t the marginal γ_t is obtained by querying the cluster graph for the posterior over X_t and reducing for each cell value:

```

for (int t = 0; t ≤ lastT; ++t) {
    rcptr<Factor> q = queryLBP_CG(*cgPtr, msgs, {X[t]})->normalize();
    for (int j = 0; j < G; ++j) {

```

```

rcptr<Factor> reduced = q->observeAndReduce({X[t]}, {T(j)});
gammas[t][j] = reduced->potentialAt(
    emdw::RVIds{}, emdw::RVVals{}, true);
}
}

```

In `miniproject_q3.cc` the E-step code is wrapped in an EM loop. After each E-step the A_t values are computed directly from `gammas[t][j]` and the detection grid, and the M-step updates (p_w, p_c) per the equation in Section (2.7):

```

double sumA = 0.0, sumDminusA = 0.0;
for (int t = 0; t ≤ lastT; ++t) {
    double At = 0.0;
    for (int row = 0; row < H; ++row)
        for (int col = 0; col < W; ++col)
            At += gammas[t][cellOf(row, col)] * static_cast<double>(detections[t][row][col]);
    sumA += At;
    sumDminusA += static_cast<double>(Dt[t]) - At;
}
double newPw = sumA / static_cast<double>(lastT + 1);
double newPc = sumDminusA / (static_cast<double>(lastT + 1) * static_cast<double>(6 - 1));
newPw = clamp(newPw, PARAM_EPS, 1.0 - PARAM_EPS);
newPc = clamp(newPc, PARAM_EPS, 1.0 - PARAM_EPS);

```

3.5 Output

For each time step the smoothed marginal is reshaped from its length- G form into a $W \times H$ matrix of probabilities and written to `marginal_<tt>.txt`; the per-step argmax cell forms a trajectory file in the ground-truth coordinate format $(x, y) = (\text{col}, \text{row})$.

4 Results and analysis

4.1 Evaluation metrics

The inferred trajectory is evaluated against the provided ground truth using:

- **Per-step accuracy:** the fraction of time steps for which $\arg \max_x \gamma_t(x)$ equals the ground-truth cell.
- **Mean cell distance:** average Euclidean distance (in cells) between the inferred argmax cell and the ground-truth cell across time steps.
- **Top-k accuracy:** the fraction of time steps for which the ground-truth cell lies in the k most probable cells under γ_t , reported for $k = 1, 3, 5$.

The evaluation uses posterior heatmaps for representative time steps with the ground-truth cell overlaid.

4.2 Dataset 1 (5×5 , $p_w=0.95$, $p_c=0.05$)

Dataset 1 was the simplest, with a small grid, and the argmax output was the same as the ground truth for every timestep. See image 2 for the posterior heatmaps of the first 4 frames, the posterior probability was very certain of all wumpus positions for all timesteps.

4.3 Dataset 2 (20×20 , $p_w=0.9$, $p_c=0.1$)

The larger grid of dataset 2 had slightly noisier detections, but except for in cases where 2 cells were almost equally likely, the argmax was still correct. For example, in the first frame there was not enough information to know whether the wumpus was in cell [15, 16] ($p = 0.493$) or [16, 17] ($p = 0.487$) See figure 3 for the posterior heatmaps of the first 4 frames.

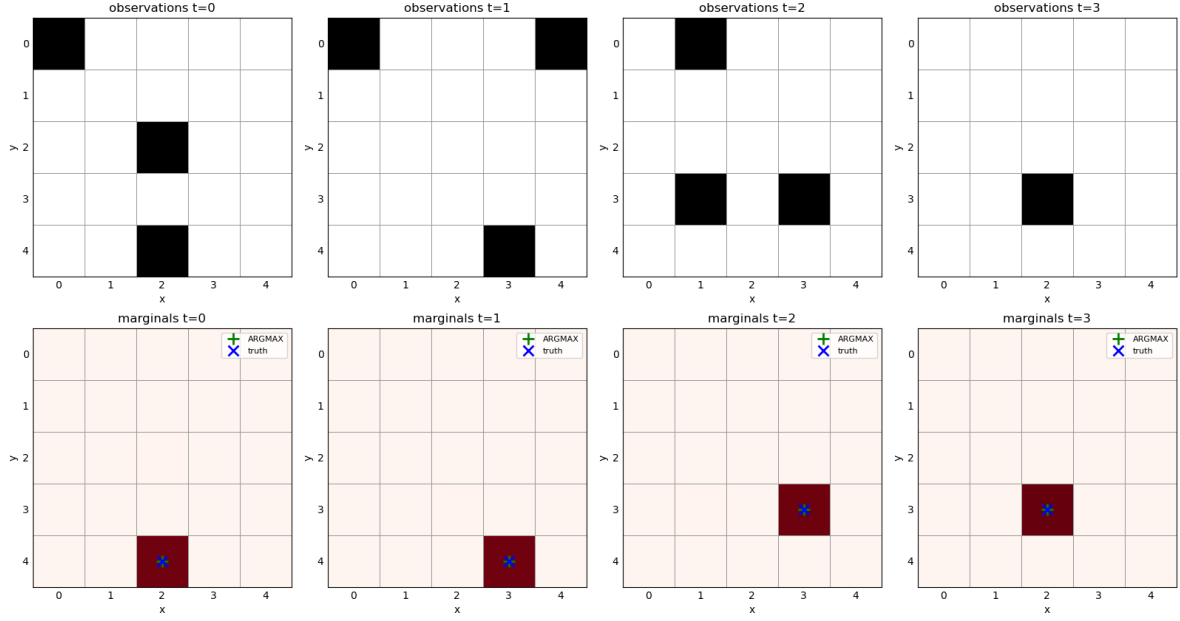


Figure 2: Posterior heatmaps for dataset 1 at the first 4 time steps; ground-truth cell marked with a blue cross, the inferred argmax cell with a green cross and the posterior probability for each cell as the heatmap.

4.4 Dataset 3 (10×20 , EM-learned parameters)

The accuracy on dataset 3 was the worst, however the initial observation frames were highly uninformative due to the noisy observations being in places that the wumpus could conceivably be, and therefore the certainty in the first 4 frames was very low and inconsistent with the ground truth. See figure 4 for the posterior heatmaps of the first 6 frames. After frame 4, the observations were more informative and the posterior certainty increased, however the argmax was still incorrect for time step 6, where the wumpus could have moved to 2 different positions with almost equal probability.

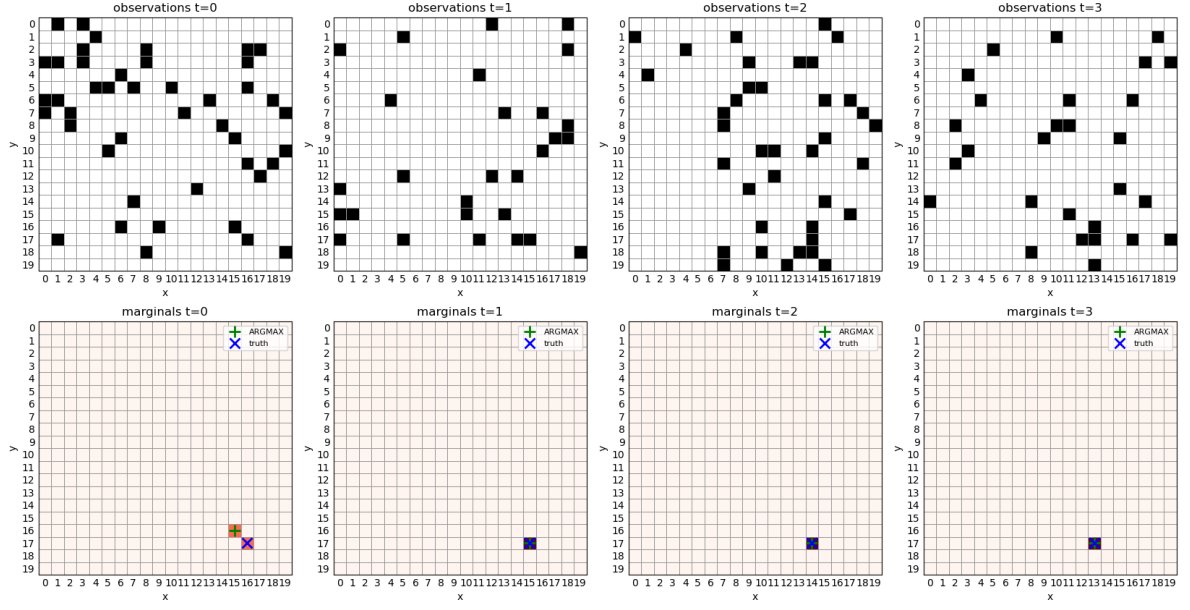


Figure 3: Posterior heatmaps for dataset 2 at the first 4 time steps; ground-truth cell marked with a blue cross, the inferred argmax cell with a green cross and the posterior probability for each cell as the heatmap.

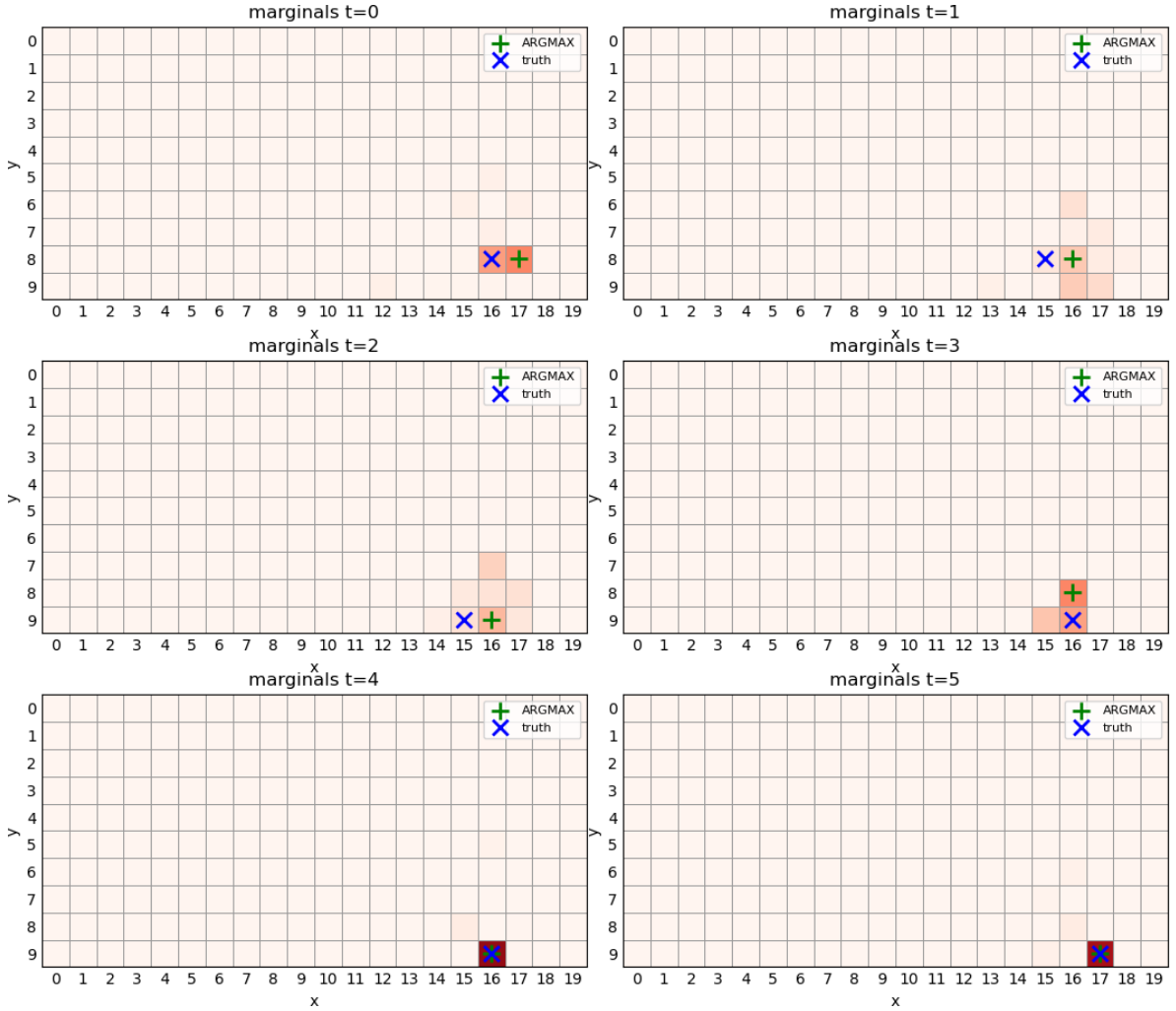


Figure 4: Posterior heatmaps for dataset 3 at the first 6 time steps; ground-truth cell marked with a blue cross, the inferred argmax cell with a green cross and the posterior probability for each cell as the heatmap.

Dataset	Per-step accuracy	Mean cell distance	Top-3 acc.	Top-5 acc.
1	100.0%	0.00	100.0%	100.0%
2	85.0%	0.21	100.0%	100.0%
3	80.0%	0.20	90.0%	100.0%

Table 1: Accuracy results across the three datasets.

```

Starting EM (max 50 iterations, tol = 1e-06)
iter 0: pw = 0.90000, pc = 0.10000 -> pw = 0.82685, pc = 0.07600 (delta = 7.31e-02, 2.40e-02)
iter 1: pw = 0.82685, pc = 0.07600 -> pw = 0.81795, pc = 0.07604 (delta = 8.90e-03, 4.47e-05)
iter 2: pw = 0.81795, pc = 0.07604 -> pw = 0.81597, pc = 0.07605 (delta = 1.98e-03, 9.95e-06)
iter 3: pw = 0.81597, pc = 0.07605 -> pw = 0.81553, pc = 0.07605 (delta = 4.46e-04, 2.24e-06)
iter 4: pw = 0.81553, pc = 0.07605 -> pw = 0.81543, pc = 0.07605 (delta = 1.01e-04, 5.06e-07)
iter 5: pw = 0.81543, pc = 0.07605 -> pw = 0.81541, pc = 0.07605 (delta = 2.28e-05, 1.14e-07)
iter 6: pw = 0.81541, pc = 0.07605 -> pw = 0.81540, pc = 0.07605 (delta = 5.14e-06, 2.58e-08)
iter 7: pw = 0.81540, pc = 0.07605 -> pw = 0.81540, pc = 0.07605 (delta = 1.16e-06, 5.84e-09)
iter 8: pw = 0.81540, pc = 0.07605 -> pw = 0.81540, pc = 0.07605 (delta = 2.63e-07, 1.32e-09)

EM converged after 9 iterations.
Final parameters: pw = 0.81540, pc = 0.07605

```

Figure 5: Convergence of p_w and p_c on dataset 3.

EM converged after 9 iterations to $(\hat{p}_w, \hat{p}_c) = (0.815, 0.076)$ with an epsilon of 10^{-6}

4.5 Cluster-graph choice

Comparing the LTRIP, BETHE and JTREE inference on dataset 3, over 25 iterations, the mean time for LTRIP was lowest at 0.5s, with JTREE at 0.55s and BETHE at 0.65s, all with negligible difference in time variance. In a situation where inference about the current position of the wumpus is necessary as fast as possible, and there is a significant amount of data, LTRIP would be the best choice. If the wumpus is doing what it was originally designed for by Gregory Yob (being hunted), faster inference of the wumpus would be more accurate, because the wumpus is running away (and thus changing its position) as its position is being inferred.

5 Conclusion

The HMM-Bayes net approach was successful in inferring the wumpus position accurately most of the time, and where it was incorrect it was incorrect but unsure of its correctness. The EM algorithm was able to learn the parameters for dataset 3, however the accuracy was worse than datasets 1 and 2, which is a result that would require further investigation to fix. The cluster graph choice did not have a significant effect on the results, however LTRIP was slightly faster than JTREE and BETHE. An interesting observation is that the accuracy of inference using miniproject.cc with dataset 4, and miniproject_q3.cc with dataset 5 resulted in accuracies of 89% and 84.5% respectively. This, while not achieving the occupied cell learning, shows that in a real world situation where the wumpus is being tracked, learning the occupied cells would be more accurate, but using these simpler models might be a worthwhile trade-off in the time-sensitive application.

References

- [1] D. Barber, *Bayesian Reasoning and Machine Learning*. Cambridge, U.K.: Cambridge University Press, 2012.
- [2] J. A. du Preez and C. E. van Daalen, “DE424 probabilistic graphical models: Course notes,” Department of Electrical and Electronic Engineering, Stellenbosch University, Stellenbosch, South Africa, Tech. Rep., 2026.
- [3] J. A. du Preez and C. E. van Daalen, “emdw user guide,” Department of Electrical and Electronic Engineering, Stellenbosch University, South Africa, unpublished technical document.